Lab 2: Data Versioning with DVC and API Ingestion

- **Course:** Engineering of Intelligent Models (MLOps & LLMOps)
- **Module:** M1 – MLOps Foundations
- **Focus Tool:** DVC & Open-Meteo API
- **Format:** Hands-on Laboratory
- **Branch:** `Lab2`

1. Goal of the Lab

The objective of this lab is to move away from static, manual CSV management and implement an engineering approach to the data lifecycle. In Machine Learning, your "source" is not just code: it is the combination of code and the exact state of the data at the time of training. You will learn how to:

1. **Initialise DVC** to track entire directories, including datasets and model weights.
2. **Implement professional ingestion** using the Open-Meteo API with unique timestamps for every run.
3. **Understand the Metafile (.dvc)** and how it creates a "bridge" between Git's small text tracking and DVC's large binary storage.
4. **Integrate DVC hashes into MLflow** to ensure absolute traceability across your pipeline.

2. Version Control: Switching Branches

Before proceeding, ensure your workspace is clean. It is a best practice to keep each lab's progress on a separate branch for easy comparison.

Switch from `Lab1` to a new branch for this session.

In [ ]:
```
# Create and switch to the Lab2 branch
!git checkout -b Lab2
```

3. Initialising DVC for Directory Tracking

DVC is not just a storage tool; it is a reproducibility framework. While Git tracks your source code, DVC will track the contents of your `data/` and `models/` folders. Crucially, **DVC can track any large file**, including model weights (e.g., `.pkl`, `.h5`, `.onnx`), preventing your Git repository from becoming bloated.

Initialise DVC and configure a local remote (this simulates an external cloud bucket like S3). You can check DVC documentation for more details on setting up different types of remotes (e.g., S3, GCS, Azure) in here https://doc.dvc.org/user-guide/data-management/remote-storage.

> **Note**: Do not forget to add DVC to your `requirements.txt` and install it in your environment if you haven't already.

In [ ]:
```
# For me, I need to change the current working directory to the project root ('EMI-WeatherFo
import os
os.chdir('../../')  # Change to project root if notebook is in a subfolder
```

In [ ]:
```
# Initialise DVC in the project root
!dvc init

# Setup a local directory as a 'remote' storage to simulate an external bucket
```

```
# If you're on Windows, use backslashes. On Unix-based systems, use forward slashes: mkdir -
!mkdir tmp\dvc_remote
!dvc remote add -d local_storage tmp/dvc_remote

# Commit DVC internal configuration to Git
!git add .dvc .dvcignore
!git commit -m "Initialize DVC with local remote storage"
```

What is a DVC "Remote"?

In your project, Git only knows about the tiny `.dvc` pointer files (metadata). It has no idea where the actual multi-gigabyte CSV files are (and also, Git cannot handle large files).

By setting up a remote:

- **Git** tracks the "recipe" (the `.dvc` file).
- **DVC Remote** tracks the "ingredients" (the actual data blobs). Without a remote, if you deleted your local `data/` folder and your local DVC cache, your data would be gone forever, even if you had the Git commits.

In a production environment, you would never store datasets on your local hard drive alone. You would use **Amazon S3, Google Cloud Storage, or Azure Blob Storage**.

- The folder `/tmp/dvc_remote` acts as a **local mock** of these cloud services.
- It allows you to practice the `dvc push` and `dvc pull` commands, which are the primary way team members sync data without passing around massive files via email or USB.

If a colleague clones my `EMI-WeatherForecast` repo, they will get your code and the `.dvc` files, but the `data/` folder will be empty.

- Because I configured a remote and, If I ran `dvc push`, your colleague can simply run `dvc pull`.
- DVC will look at the hash in the `.dvc` file, go to the remote folder, and download the exact version of the weather data needed for that specific Git branch.

4. Dynamic Data Ingestion with Timestamps

In production systems, data arrives continuously. Overwriting a single `weather.csv` file is dangerous because it destroys historical context. We will save data with **ISO-8601 timestamps**, allowing DVC to track the arrival of new files automatically.

Update your `requirements.txt` and make sure you `pip install` them before running the ingestion script.

```
mlflow==3.10.0
pandas==2.3.3
dvc==3.66.1
openmeteo-requests==1.7.5
requests-cache==1.3.0
retry-requests==2.0.0
pyyaml==6.0.3
```

Create `src/ingestion/get_data.py` and paste the following logic:

> **NOTE**: This script retrieves hourly weather data (`temperature_2m`, `relative_humidity_2m`, `precipitation`) for a specific location (I chose Sintra 😅) and time range, processes it into a DataFrame, and saves it as a CSV file with a unique timestamp in the filename. The use of `requests_cache` and `retry` ensures that API calls are efficient and resilient to transient failures.

```python
import openmeteo_requests
import pandas as pd
import requests_cache
from retry_requests import retry
from datetime import datetime
import os


def fetch_weather_data():
    # Setup API client with cache to avoid redundant calls
    cache_session = requests_cache.CachedSession('.cache', expire_after=-1)
    retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
    openmeteo = openmeteo_requests.Client(session=retry_session)

    # API Parameters for the Weather Forecast Project
    url = "https://archive-api.open-meteo.com/v1/archive"
    params = {
        "latitude": 38.801,  # Sintra, Portugal
        "longitude": -9.3783,
        "start_date": "2026-02-06",
        "end_date": "2026-02-20",
        "hourly": ["temperature_2m", "relative_humidity_2m", "precipitation"],
        "timezone": "auto",
    }

    print(f"Requesting weather data from Open-Meteo...")
    responses = openmeteo.weather_api(url, params=params)
    response = responses[0]  # Assuming we only have one response for the given
parameters

    # Process hourly data into a dictionary
    hourly = response.Hourly()
    hourly_data = {
        "date": pd.date_range(
            start=pd.to_datetime(hourly.Time() + response.UtcOffsetSeconds(),
unit="s", utc=True),
            end=pd.to_datetime(hourly.TimeEnd() + response.UtcOffsetSeconds(),
unit="s", utc=True),
            freq=pd.Timedelta(seconds=hourly.Interval()),
            inclusive="left"
        ),
        "temperature_2m": hourly.Variables(0).ValuesAsNumpy(),
        "relative_humidity_2m": hourly.Variables(1).ValuesAsNumpy(),
        "precipitation": hourly.Variables(2).ValuesAsNumpy()
    }

    df = pd.DataFrame(data=hourly_data)

    # Generate timestamped filename for automatic capturing
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"weather_{timestamp}.csv"
    os.makedirs("data/raw", exist_ok=True)
    output_path = os.path.join("data/raw", filename)

    df.to_csv(output_path, index=False)
    print(f"Data ingested and saved to: {output_path}")


if __name__ == "__main__":
    fetch_weather_data()
```

5. Tracking the Entire Data Folder

Instead of tracking individual files, we will tell DVC to track the `data/raw` folder as a single entity. When new timestamped files arrive, running `dvc add data/raw` will update the folder's unique hash.

Execute the ingestion and hand over control to DVC.

In [ ]:
```
# First we need to remove 'data/raw' from Git tracking if it was added in Lab1
!git rm -r --cached data/raw
!git commit -m "Stop tracking data/raw"
```

In [ ]:
```
# 1. RUN ingestion
!python src/ingestion/get_data.py
```

In [ ]:
```
# 2. ADD the whole folder to DVC (calculates hash)
!dvc add data/raw
```

In [ ]:
```
# 3. COMMIT the metadata to Git
!git add data/raw.dvc
!git commit -m "Track raw data folder with DVC"
```

In [ ]:
```
# 4. PUSH the actual data to your Remote (/tmp/dvc_remote)
!dvc push
```

Git vs DVC: CLI commands analogy

Command	Action	Analogous to
`dvc add`	Moves data to local cache and creates `.dvc` pointer.	`git add`
`dvc push`	Uploads data from local cache to Remote Storage.	`git push`
`dvc pull`	Downloads data from Remote Storage to local workspace.	`git pull`

6. Updating MLflow: Tracking Data Lineage

We will now update our connectivity test to capture the **DVC Hash** of the data folder. This creates a permanent link between your experiment and the exact state of your data. If you ever need to reproduce this exact run, you can use the hash to revert your data folder to this specific state.

Create `src/training/lab2_dvc_integration.py`:

```
import mlflow
import pandas as pd
import dvc.api
import yaml
import os

mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("Lab2_DVC_Integration")


def dvc_integration():
    # Use the DVC API to get the internal URL/Hash of the tracked folder
    try:
        # get_url returns the path in the .dvc/cache (which is named by the MD5
hash)
        resource_url = dvc.api.get_url(path='data/raw', repo='.')
        folder_hash = os.path.basename(resource_url)

    except Exception as e:
        print(f"Error accessing DVC API: {e}")
        return
```

```
    with mlflow.start_run(run_name="Lab2_DVC_Versioning") as run:
        # Select the latest file for the current run
        files = [f for f in os.listdir("data/raw") if f.endswith(".csv")]
        latest_file = sorted(files)[-1]
        df = pd.read_csv(os.path.join("data/raw", latest_file))

        # Log the DVC Hash for Data Lineage
        # This provides a 1:1 link between this MLflow run and the DVC data state
        mlflow.log_param("dvc_data_hash", folder_hash)
        mlflow.log_param("input_file", latest_file)
        mlflow.log_metric("row_count", len(df))
        mlflow.log_metric("average_temperature", df["temperature_2m"].mean())
        mlflow.log_metric("average_relative_humidity",
df["relative_humidity_2m"].mean())
        mlflow.log_metric("average_precipitation", df["precipitation"].mean())

        print(f"Logged successfully!")
        print(f"DVC Data Hash extracted via API: {folder_hash}")


if __name__ == "__main__":
    dvc_integration()
```

> **NOTE**: The use of `dvc.api.read` allows us to programmatically access the metadata of our tracked folder. By logging the `md5` hash to MLflow, we create a direct link between our experiment and the exact version of the data used. This is crucial for reproducibility, as it allows us to revert anytime to the specific state of the data that was used for that run, even if new files have been added since then.

Run the script to see the integration in action:

In [ ]:
```
!python src/training/lab2_dvc_integration.py
```

7. Lab Checklist

1. Inspect `/tmp/dvc_remote`. Do you see hashed folders representing your data?
2. Does the `lab2_dvc_integration.py` correctly extract the `md5` using `dvc.api.read`?
3. In the MLflow UI, can you verify if dataset versioning is working? It should be available as a parameter in the run details (`dvc_data_hash`).
    - This hash is your "data fingerprint" for that specific run. If you ever need to reproduce this run, you can use this hash to revert your `data/raw` folder to the exact state it was in at the time of training, ensuring perfect reproducibility even as new data arrives.

8. Next Steps

Your data is versioned, shared on a remote, and programmatically linked to your experiments. In the next lab, we will integrate Hydra and remove the hardcoded API coordinates and date ranges. This will allow us to run the same ingestion logic for any location and time period, making our pipeline truly dynamic and reusable.